Kubernetes Assignment: Zero-Downtime Releases at ShopKart

You are a Junior DevOps Engineer at ShopKart, and you must ship three releases of the storefront without ever taking the site down.

Scenario

ShopKart is an online store running on Kubernetes. The storefront is a stateless web app; for this lab it is played by the `nginx` image, so the version number (nginx 1.25 vs 1.26) stands in for your app version. Your manager has given you three releases, each with a different risk level:

Release	Risk	Required strategy
Minor patch (nginx 1.25 to 1.26)	Low	RollingUpdate, zero downtime
Big checkout redesign	High: must be tested on real infrastructure before any customer sees it	Blue/green
New recommendation engine	Unknown impact: expose to a few customers first	Canary (about 10% of traffic)

At the end, a bad release is pushed by mistake, and you must contain it and recover.

Learning objectives

- Write Deployment and Service manifests from scratch and apply them declaratively.
- Use labels and selectors to control which Pods receive traffic.
- Implement blue/green by switching a Service selector, and roll back instantly.
- Implement a canary by splitting replicas between two Deployments behind one Service.
- Measure traffic with a curl loop instead of guessing, and justify which strategy fits which risk.

Ground rules

- Work in a new folder: `mkdir k8s-assignment && cd k8s-assignment`. Every object must come from a YAML file you wrote; no `kubectl create deployment` or `kubectl run` for the app (the `tester` helper Pod is the one exception).
- Pin image tags (`nginx:1.25`, `nginx:1.26`). Never use `latest`.
- Use the `default` namespace on Docker Desktop. Run `kubectl delete all --all` between parts, then re-create `tester`.
- Use the `tester` Pod from class for all traffic tests: `kubectl run tester --image=curlimages/curl --restart=Never -- sleep 3600`.
- To tell versions apart, read the `Server` header: `kubectl exec tester -- curl -sI --max-time 2 <service> | grep -i ^server` prints `nginx/1.25.x` or `nginx/1.26.x`.
- Save screenshots or pasted terminal output as evidence for every task marked **Evidence**.

Hint: if a Service shows `<none>` endpoints, compare its selector with the Pod labels. If NodePort does not respond on Docker Desktop, use `kubectl port-forward svc/<name> 8080:80`. If Pods stay Pending, lower replicas or requests.

Part 1: Release 1, the safe rolling update

ShopKart is live on `nginx:1.25`. Ship the low-risk patch without dropping a single request.

1. Write `shopkart.yaml` containing a Deployment named `shopkart` (4 replicas, label `app: shopkart`, container `web`, image `nginx:1.25`) and a ClusterIP Service `shopkart-svc` on port 80. Use `RollingUpdate` with `maxSurge: 1` and `maxUnavailable: 0`, a readiness probe on `/`, and CPU/memory requests and limits.
2. Apply it and confirm 4/4 Pods are Ready and `kubectl get endpoints shopkart-svc` shows 4 addresses.
3. Start a curl loop in `tester` that prints the HTTP status twice a second, and a Pod watcher in another terminal.
4. Update the image to `nginx:1.26` by editing the YAML and running `kubectl apply` (not `set image`). Add a `kubernetes.io/change-cause` annotation.
5. Run `kubectl rollout history` and `kubectl get rs`.

Hint: run `kubectl apply -f shopkart.yaml --dry-run=client` and `kubectl diff` before applying. Start the loop only after all 4 Pods show READY 1/1, and check `kubectl describe deployment shopkart` for the strategy and `Readiness:` lines. `kubectl rollout status` tells you when the update has finished.

Evidence: the manifest, the loop output during the update, and `kubectl get rs` showing old RS at 0 and new RS at 4.

Answer in 3 to 4 sentences: Why did the loop stay at 200? What would have changed if you had used `Recreate`?

Part 2: Release 2, blue/green for the checkout redesign

The redesign must be fully running and testable on the cluster before customers touch it, and you need a one-step rollback. Blue is the current version (`nginx:1.25`); green is the new one (`nginx:1.26`).

Setup

1. Delete Part 1 objects. Write `blue.yaml`: Deployment `shopkart-blue`, 3 replicas, image `nginx:1.25`, Pod labels `app: shopkart` and `version: blue`. The `selector.matchLabels` must include both labels.
2. Write `service.yaml`: NodePort Service `shopkart-live` (nodePort 30080) whose selector is `app: shopkart` and `version: blue`.
3. Apply both. Confirm `curl http://localhost:30080` works (or use `tester` against `shopkart-live` if NodePort fails on your machine) and the Server header shows 1.25.

Hint: the Pod template labels must satisfy `selector.matchLabels`, so put both `app` and `version` in both places. Use `kubectl get pods --show-labels` to confirm what each Pod really carries.

Deploy green beside blue

4. Write `green.yaml`: Deployment `shopkart-green`, 3 replicas, image `nginx:1.26`, labels `app: shopkart` and `version: green`. Apply it.
5. Write `preview-service.yaml`: NodePort Service `shopkart-preview` (nodePort 30081) selecting `version: green`. Use it to test green while `shopkart-live` still serves only blue. Prove this with two curl commands, one against each Service.
6. Check `kubectl get endpoints` for both Services. The two lists must contain different Pod IPs.

Hint: `kubectl get endpoints shopkart-live shopkart-preview -o wide` lets you compare both lists in one command. Two different sets of Pod IPs means your selectors work.

Switch and roll back

7. Start the status/Server-header loop against `shopkart-live`. Edit `service.yaml`, change `version: blue` to `version: green`, and apply. Record what the loop shows at the moment of the switch.
8. Simulate a failure report from the business: switch the selector back to `blue` and apply. Compare how long this rollback took with `kubectl rollout undo` from class.
9. Switch to green again and, once you are satisfied, scale `shopkart-blue` to 0 replicas in its YAML (keep the file for the next release).

Hint: only the Service changes during the switch, so `kubectl get pods` should look identical before and after. Run `kubectl describe svc shopkart-live` to see the selector the Service is using right now.

Evidence: all four manifests, both endpoint listings, the loop output across both switches.

Answer:

- Why is the switch effectively instant, and why does it need no rollout at all?
- What does blue/green cost that a rolling update does not (think of capacity during step 5)?
- Give one situation where blue/green is clearly better than a rolling update.

Part 3: Release 3, canary for the recommendation engine

Nobody knows how the new engine behaves under real traffic. Send it to about 10% of requests first, then widen. Stable is `nginx:1.25`; canary is `nginx:1.26`.

Setup

1. Delete Part 2 objects. Write `stable.yaml`: Deployment `shopkart-stable`, 9 replicas, image `nginx:1.25`, Pod labels `app: shopkart` and `track: stable`. If your single-node cluster struggles, use 3 stable and 1 canary replicas (25%) and say so in your report.
2. Write `canary.yaml`: Deployment `shopkart-canary`, 1 replica, image `nginx:1.26`, labels `app: shopkart` and `track: canary`.

3. Write `service.yaml`: ClusterIP Service `shopkart-svc` whose selector is **only** `app: shopkart`, so it matches both Deployments. Each Deployment's own `selector.matchLabels` must still include `track`, otherwise the two Deployments will fight over each other's Pods.
4. Confirm `kubectl get endpoints shopkart-svc` lists 10 Pod IPs.

Hint: the Service selector has one label (`app: shopkart`); the Deployment selectors have two (`app` and `track`). If `kubectl get pods` shows the wrong number of Pods, the Deployment selectors overlap.

Measure the split

5. Run 100 requests from `tester` and count versions:

```
kubectl exec tester -- sh -c 'for i in $(seq 1 100); do curl -sI --max-time 2
shopkart-svc | grep -i ^server; done' | sort | uniq -c
```

6. Repeat the test three times and record the counts in a table (version, run 1, run 2, run 3). Traffic is spread randomly by kube-proxy, so expect roughly 90/10, not exactly.

Hint: 100 requests give a noisy answer; run 200 if your results swing a lot. `sort | uniq -c` at the end of the pipeline does the counting for you.

Promote step by step

7. The canary looks healthy. Promote in stages by editing replicas and applying: 7 stable and 3 canary, then 5 and 5, then 0 stable and 10 canary. Re-run the 100-request test at each stage and record the split.
8. Update `stable.yaml` to `nginx:1.26`, scale it to 10, delete `shopkart-canary`, and confirm the final state serves only 1.26.

Abort path

9. Repeat steps 1 to 5, but this time the canary image is `nginx:9.99` (a broken release). Observe the Pod status, then check what the 100-request test shows. Explain why customers are barely affected, and what the abort action is.

Hint: a Pod stuck in `ImagePullBackOff` is not Ready, so it is removed from the Service endpoints. Check `kubectl get endpoints shopkart-svc` and `kubectl describe pod <name>`. The abort is scaling the canary Deployment to 0 or deleting it.

Evidence: all manifests, the endpoint listing, your results table, and the output of step 9.

Answer:

- Why can you control the traffic split with replica counts alone?
- What is the smallest percentage you can send to a canary with 4 total Pods? What does that tell you about the limits of this method?
- Why does a readiness probe matter on the canary Deployment? Add one and compare step 9 with and without it.

Part 4: The 2 a.m. incident and strategy review

Incident. A teammate runs `kubectl set image deployment/shopkart-stable web=nginx:9.99` on the production Deployment from Part 3 (with `RollingUpdate`, `maxUnavailable: 0`).

1. Reproduce this. Capture `kubectl rollout status --timeout=30s`, the Pod list, and a status loop proving users are unaffected.
2. Recover with `kubectl rollout undo` and prove which revision is running with `kubectl rollout history`.
3. Now set `strategy: Recreate` on the same Deployment, repeat the bad update, and record the seconds of downtime in the loop.

Hint: `kubectl rollout history deployment/shopkart-stable --revision=<n>` shows which image each revision used. With `Recreate`, run the loop with `--max-time 1` as in class and count the `000` lines, then divide by 2 for seconds.

Strategy review. Fill in this decision table in your own words (one short sentence per cell):

	Rolling update	Blue/green	Canary
How traffic moves to the new version			
Extra capacity needed			
Rollback speed and method			
Two versions live together?			
Best use case			

Recommendation (150 to 200 words). ShopKart now has three more releases: (a) a security patch that must go out today, (b) a database schema change that the old app version cannot read, (c) a new pricing algorithm whose effect on revenue is unknown. For

each, pick a strategy (Recreate counts as an option) and justify it. Explain how you would stop a bad release in each case.

Hints: if a Service shows `<none>` endpoints, compare its selector with the Pod labels. If NodePort does not respond on Docker Desktop, use `kubectl port-forward svc/<name> 8080:80`. If Pods stay Pending, lower replicas or requests.